Disjoint-set Data Structure

Disjoint-set Data Structure

- Also known as “union find.”
- Maintain collection $\mathcal{S} = \{S_1, \ldots, S_k\}$ of disjoint dynamic (changing over time) sets.
- Each set is identified by a *representative*, which is some member of the set.

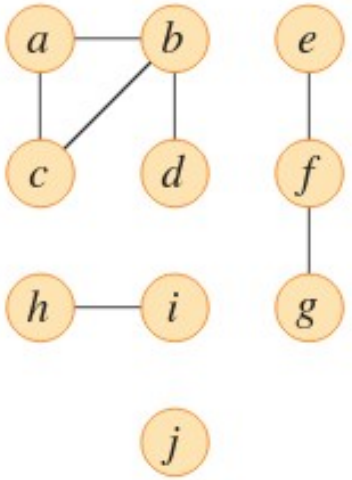
  Doesn’t matter which member is the representative, as long as if you ask for the representative twice without modifying the set, you get the same answer both times.

Maintains a collection of disjoint sets under the operations UNION and FIND.

OPERATIONS

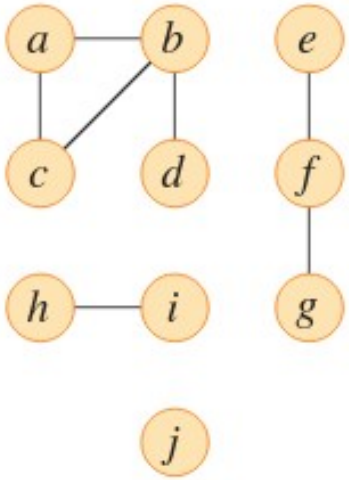
- MAKE-SET(x): make a new set $S_i = \{x\}$, and add S_i to $\mathcal{S}$.
- UNION(x, y): if $x \in S_x$, $y \in S_y$, then $\mathcal{S} = \mathcal{S} - S_x - S_y \cup \{S_x \cup S_y\}$.
  - Representative of new set is any member of $S_x \cup S_y$, often the representative of one of S_x and S_y.
  - Destroys S_x and S_y (since sets must be disjoint).
- FIND-SET(x): return representative of set containing x.

#Connected Components



Edge processed
initial sets
(b, d)
(e, f)
(a, c)
(h, i)
(a, b)
(f, g)
(b, c)

#Connected Components



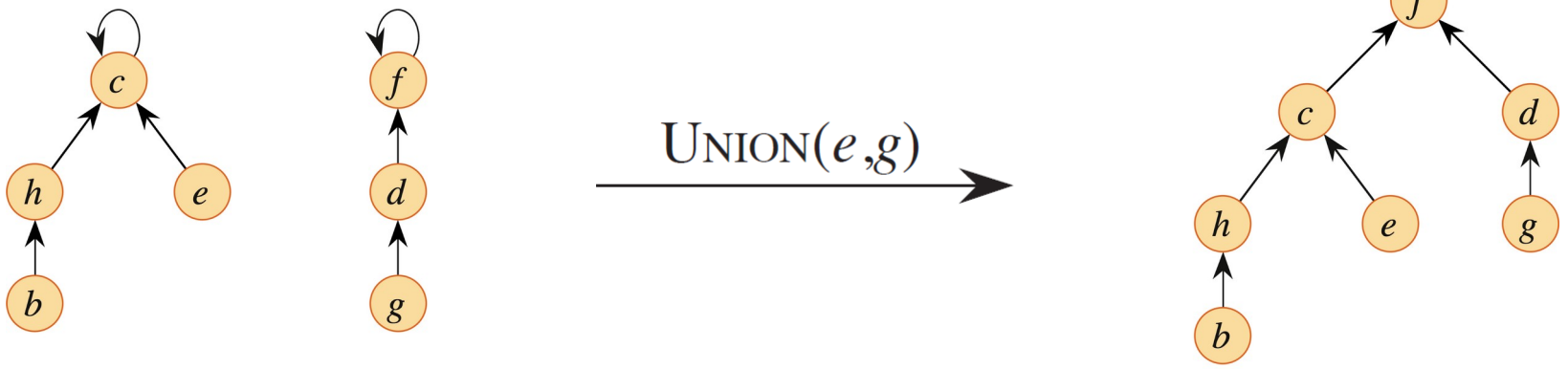
Edge processed	Collection of disjoint sets									
initial sets	$\{a\}$	$\{b\}$	$\{c\}$	$\{d\}$	$\{e\}$	$\{f\}$	$\{g\}$	$\{h\}$	$\{i\}$	$\{j\}$
(b,d)	$\{a\}$	$\{b,d\}$	$\{c\}$		$\{e\}$	$\{f\}$	$\{g\}$	$\{h\}$	$\{i\}$	$\{j\}$
(e,f)	$\{a\}$	$\{b,d\}$	$\{c\}$		$\{e,f\}$		$\{g\}$	$\{h\}$	$\{i\}$	$\{j\}$
(a,c)	$\{a,c\}$	$\{b,d\}$			$\{e,f\}$		$\{g\}$	$\{h\}$	$\{i\}$	$\{j\}$
(h,i)	$\{a,c\}$	$\{b,d\}$			$\{e,f\}$		$\{g\}$	$\{h,i\}$		$\{j\}$
(a,b)	$\{a,b,c,d\}$				$\{e,f\}$		$\{g\}$	$\{h,i\}$		$\{j\}$
(f,g)	$\{a,b,c,d\}$				$\{e,f,g\}$			$\{h,i\}$		$\{j\}$
(b,c)	$\{a,b,c,d\}$				$\{e,f,g\}$			$\{h,i\}$		$\{j\}$

DISJOINT-SET FOREST

Running time: ??

Forest of trees.

- 1 tree per set. Root is representative.
- Each node points only to its parent.



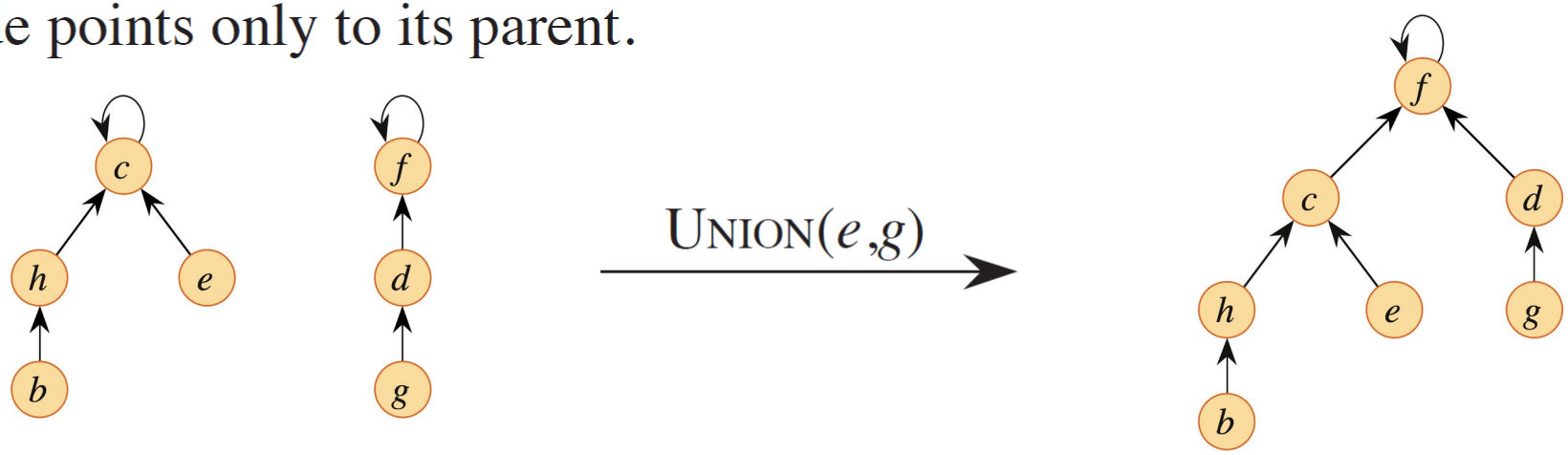
- MAKE-SET: make a single-node tree.
- UNION: make one root a child of the other.
- FIND-SET: follow pointers to the root.

DISJOINT-SET FOREST

Running time:
O(n . U + m . F)

Forest of trees.

- 1 tree per set. Root is representative.
- Each node points only to its parent.



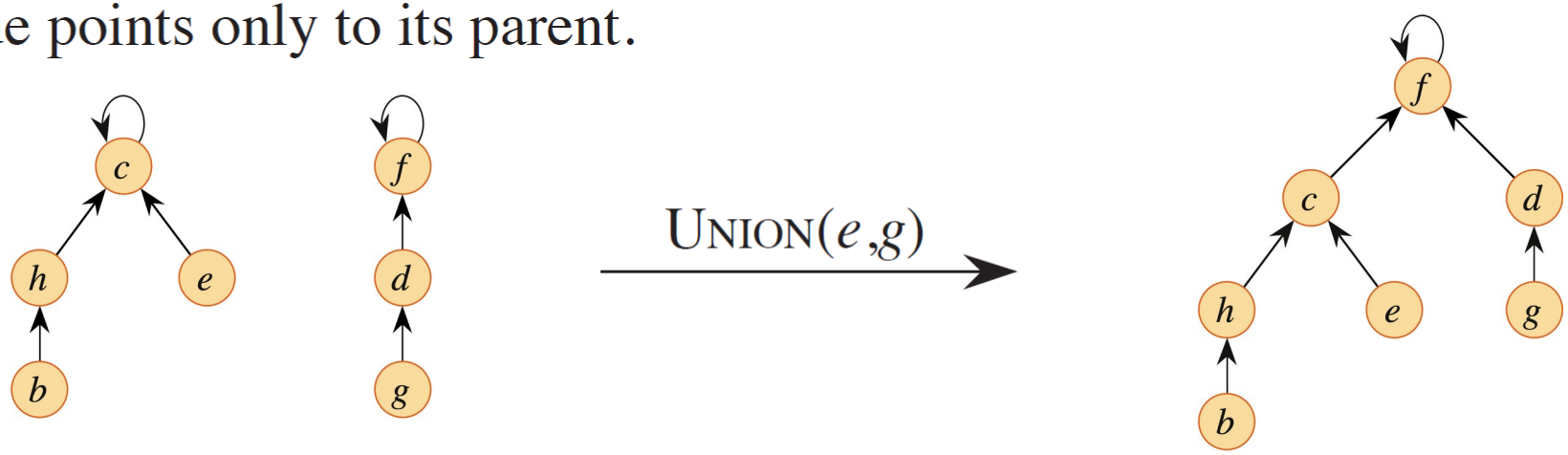
- MAKE-SET: make a single-node tree.
- UNION: make one root a child of the other.
- FIND-SET: follow pointers to the root.

DISJOINT-SET FOREST

Running time:
O(n . U + m . F)

Forest of trees.

- 1 tree per set. Root is representative.
- Each node points only to its parent.



- MAKE-SET: make a single-node tree.
- UNION: make one root a child of the other.
- FIND-SET: follow pointers to the root.

Not so good—could get a linear chain of nodes.

Running time:
O(n + **m n**)

Efficient UNION

UNION BY RANK

- Two trees T_1 and T_2, with nodes n_1 and n_2 nodes.

Make the lighter tree point to the heavier tree.

Consequence: Height is under control. Why?

- ***Union by rank:*** make the root of the smaller tree (fewer nodes) a child of the root of the larger tree.
  - Don't actually use *size*.
  - Use *rank*, which is an upper bound on height of node.
  - Make the root with the smaller rank into a child of the root with the larger rank.

Efficient UNION

UNION BY RANK

Theorem: A tree with n_1 nodes has height at most log n_1.

Proof by induction on the number of nodes.

Consider a union of trees with n_1 and n_2 nodes.

Height of the resultant tree is $h = \max\{h_2, h_1+1\}$

Efficient UNION

UNION BY RANK

Theorem: A tree with n nodes has height at most log n.

Proof by induction on the number of nodes.

Consider a union of trees with n_1 and n_2 nodes.

Height of the resultant tree is $h = \max\{h_2, h_1+1\}$

Case 1: $h=h_2$ implies $h \leq \log n_2 \leq \log (n_1+n_2)$.

Case 2: $h=h_1+1$ implies $h \leq (\log n_1) + 1 = \log 2n_1 \leq \log (n_1+n_2)$.

Efficient UNION

- Two trees T_1 and T_2, with nodes n_1 and n_2 nodes.
  Make the **shallow** tree point to the **taller** tree.

Consequence: Height is under control. Why?

Theorem:

A tree of height h has at least $\mathbf{2^h}$ nodes.

UNION BY HEIGHT

Proof by Induction

UNION BY HEIGHT

Efficient UNION

Theorem: A tree of height h has at least 2^h nodes.

Proof by Induction on the height.

Consider a union of trees with heights h_1 and h_2. The number of nodes are n_1 and n_2, where $n_1 \geq 2^{h1}$ and $n_2 \geq 2^{h2}$.

Height of the resultant tree is $h = \max\{h_2, h_1+1\}$

UNION BY HEIGHT

Efficient UNION

Theorem: A tree of height h has at least 2^h nodes.

Proof by Induction on the height.

Consider a union of trees with heights h_1 and h_2. The number of nodes are n_1 and n_2, where $n_1 \geq 2^{h1}$ and $n_2 \geq 2^{h2}$.

Height of the resultant tree is $h = \max\{h_2, h_1+1\}$

Number of nodes $= n_1+n_2 \geq 2^{h1} + 2^{h2} \geq 2^{h2}$

$\geq 2^{h1} + 2^{h1} \geq 2^{(h1+1)}$

Disjoint-set DS

Running time:
O(n . U + m . F)

Union by Rank or Union by Height

UNION: O(1)
FIND: O(log n)

Running time:
O(n + **m log n**)

$m <= n^2$

IMPROVED FIND

- ***Path compression: Find path*** = nodes visited during FIND-SET on the trip to the root. Make all nodes on the find path direct children of root.

